Vectori și valori proprii

$\lambda \in \mathbb{C}$ valoare proprie $\Rightarrow \exists\, x \in \mathbb{C}^n$ a.î.

$$Ax = \lambda x$$

$p(\lambda) = \det(A - \lambda I_n) \rightarrow$ polinom caracteristic

$p(\lambda) = 0 \iff \det(A - \lambda I_n) = 0 =$ ec. caracteristică

$\lambda(A) = \{\lambda_1, \dots, \lambda_n\} =$ spectrul lui A | eig(A)

$\rho(A) = \{\max_{\lambda_i \in \lambda(A)} |\lambda_i| =$ raza spectrală a lui A$\}$

| max (abs (eig (A)))

Prop. 1) $\sum_{i=1}^{n} \lambda_i = \sum_{i=1}^{n} A_{ii}$ | trace (A)

2) $\prod_{i=1}^{n} \lambda_i = \det(A)$ | det (A)

Norma = modalitate de a măsura mărimea unui vector, a unei matrice etc.

$$\|x\|_p = \left(\sum_{i=1}^{n} |x_i|^p\right)^{1/p}, \quad p \in \mathbb{R},\ p \geq 1,\ x \in \mathbb{R}^n$$

$p = 1 \Rightarrow \|x\|_1 = |x_1| + |x_2| + \dots + |x_n|$

↳ Distanța Manhatten

$p=2 \Rightarrow \|x\|_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$

$p=\infty \Rightarrow \|x\|_\infty = \max_{i=\overline{1,n}} |x_i|$

$$\langle x, y \rangle = \sum_{i=1}^{n} x_i y_i = y^T x$$

$$\|x\|_2 = \sqrt{\langle x, x \rangle} = \sqrt{x^T x}$$

x, y ortogonali $\iff \langle x, y \rangle = 0$

x, y ortonormați $\iff \begin{cases} \langle x, y \rangle = 0 \\ \|x\| = \|y\| = 1 \end{cases}$

Metoda puterii

$A \in \mathbb{R}^{n \times n}$

$\lambda(A) = \{ \lambda_1, \lambda_2, \dots, \lambda_n \}$

Scop → găsirea unei perechi (λ, x)

- λ: val. proprie
- x: vector propriu

u_0 = aprox. inițială vect. propriu

$u_1 = A u_0 \qquad u_{k+1} = A u_k$

$u_2 = A u_1$

$\iff \boxed{u_k = A^k \cdot u_0}$

$\vdots$

↳ vectori proprii ai lui A

Dacă $x_1, \dots, x_n$ liniar independenți

$u_0 = c_1x_1 + c_2x_2 + \dots + c_nx_n \qquad Ax_i = \lambda x_i$

$u_k = A^k u_0 = c_1A^kx_1 + c_2A^kx_2 + \dots + c_nA^kx_n$

$u_k = c_1\lambda_1^k x_1 + c_2\lambda_2^k \cdot x_2 + \dots + c_n\lambda_n^k \cdot x_n$

Presupunem că $|\lambda_1| \leq \dots \leq |\lambda_{n-1}| < |\lambda_n|$

Dacă u_0 a conținut măcar o componentă din x_n ($c_n \neq 0$) $\Rightarrow$ $\underline{c_n\lambda_n^k x_n}$ domină $\underline{u_k}$

$\frac{u_k}{\lambda_n^k} = c_1\left(\frac{\lambda_1}{\lambda_n}\right)^k x_1 + \dots + c_{n-1}\left(\frac{\lambda_{n-1}}{\lambda_n}\right)^k x_{n-1} +$

($+\infty \to 0$ pentru $\left(\frac{\lambda_1}{\lambda_n}\right)^k$; $+\infty \to 0$ pentru $\left(\frac{\lambda_{n-1}}{\lambda_n}\right)^k$)

$\boxed{c_nx_n}$ ⟶ vector propriu

$k \to +\infty \Rightarrow u_k \to x_n$

Convergență: $r = \frac{|\lambda_{n-1}|}{|\lambda_n|}$ <—> eroarea scade cu aproximativ $1-r$ la fiecare iterație

↓ dacă r e mai aproape de $0 \Rightarrow$ converge mai rapid!

=> vectorul devine mult prea lung (∞ de lung)

=> soluție: normăm u_k la fiecare pas

Au aflat x

$Ax = \lambda x \Leftrightarrow \boxed{\lambda = x^T A x}$

↳ x normat

Octave algoritm

```
function [lambda, x, err, steps] = power_method (A, x0,
                                                 tol, max_steps)

  steps = 0;
  prev_x = x0;
  while 1
    x = A * prev_x;
    x = x / norm(x);
    lambda = x' * A * x;
    err = norm ( x - prev_x);
    if (++steps == max_steps) || (err < tol)
      return;
    endif
    prev_x = x;
  endwhile
endfunction
```

MPD
Octave algorithm

Metoda puterii inverse

Scop: găsirea celei mai mici valori proprii + un vector propriu asociat

$$\begin{cases} \lambda(A) = \{\lambda_1, \dots, \lambda_n\} \\ \lambda(A^{-1}) = \{\lambda_1^{-1}, \dots, \lambda_n^{-1}\} \end{cases}$$ obs!

$$|\lambda_1| < |\lambda_2| \leq \dots \leq |\lambda_n| \iff |\lambda_1^{-1}| > |\lambda_2^{-1}| \geq \dots \geq |\lambda_n^{-1}|$$

u_0 = aproximație inițială

$$u_{k+1} = A^{-1} u_k \iff \boxed{u_k = A^{-k} u_0}$$

$$= c_1 A^{-k} x_1 + \dots + c_n A^{-k} x_n = *$$

$$\left.\begin{aligned} Ax &= \lambda x \\ x &= \lambda A^{-1} x \\ \lambda^{-1} x &= A^{-1} x \end{aligned}\right\} \Rightarrow$$

vectorii proprii ai lui A^{-1} = vectorii proprii ai lui A

$$* = \underbrace{c_1 \lambda_1^{-k} x_1}_{\text{termen dominant}} + \dots + c_n \lambda_n^{-k} x_n$$

u_k tinde la x_1 = vect. propriu asociat celei mai mici val. proprii în modul pt. A

Algoritm → același ca la met. puterii directe,

dar $u_{k+1} = A^{-1} u_k \iff A u_{k+1} = u_k$

↳ rezolvarea acestui sistem

Dacă `lambda = x' * A^-1 * x` ⇒ vom returna λ_1^{-1}

`lambda = x' * A * x` ⇒ vom returna pe λ_1

Convergență: $r = \frac{|\lambda_1|}{|\lambda_2|}$

Metoda puterii inverse deplasată

$\lambda(A) = \{\lambda_1, \dots, \lambda_n\} \Rightarrow \lambda(A - \alpha I) = \{\lambda_1 - \alpha, \dots, \lambda_n - \alpha\}$

- Dacă $\alpha \simeq \lambda_1 \Rightarrow (\lambda_1 - \alpha)^{-k}$ dominantă pentru metoda puterii inverse aplicată pe $A - \alpha I$

- Ajunge la cea mai apropiată valoare proprie de α.

Algoritm → fix ca la M.P.I, dar $A - \alpha I$ în loc de A

Dacă λ_1 e aproximat de alt algoritm, MPID găsește λ_1 super rapid.

Dacă A este simetrică putem folosi

Câtul lui Rayleigh → aproximarea în sensul celor mai mici pătrate

$$\alpha = \frac{x^t A x}{x^t x}$$ sau $\alpha = x^t A x$ = ultima valoare proprie aprox.

=> grăbirea convergenței !

Metoda puterii ortogonalizată → găsește toate perechile val-vect. proprii !

(*) $(\forall)\ A \in \mathbb{R}^{n \times n}, x, y \in \mathbb{R}^n$

$$\langle Ax, y \rangle = \langle x, A^T y \rangle$$

Matrice asemenea

$A \sim B \iff \lambda(A) = \lambda(B)$ (au aceleași valori proprii)

$(\forall)\ A, T \in \mathbb{R}^{n \times n}$ $\boxed{T^{-1} A T \sim A}$

Se aplică doar pt. A matrice simetrică.

Presupunem că $(\lambda_1 x)$ și $(\mu_1 y)$ sunt perechi valoare proprie - vector propriu cu $\lambda \neq \mu$.

$$\lambda \langle x_1 y \rangle = \langle \lambda x_1 y \rangle = \langle Ax_1 y \rangle = \langle x_1 A^T y \rangle$$

$$= \langle x_1 \underbrace{Ay} \rangle = \langle A_1 \mu y \rangle = \mu \langle x_1 y \rangle$$

$$\Rightarrow (\lambda - \mu) \cdot \langle x_1 y \rangle = 0$$

$\lambda \neq \mu \Rightarrow \langle x_1 y \rangle = 0 \Rightarrow x_1, y$ sunt ortogonali

(toți vectorii proprii corespunzători la val. proprii diferite sunt ortogonali)

A simetrică $\Rightarrow$ 

Presupunem $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$

$$u_k = A^k \cdot u_0 = \lambda_1^k \Big(c_1 x_1 + c_2 \underbrace{\left(\frac{\lambda_2}{\lambda_1}\right)^k}_{\downarrow 0} x_2 + \dots + c_n \underbrace{\left(\frac{\lambda_n}{\lambda_1}\right)^k}_{\downarrow 0} x_n \Big)$$

$\langle u_0, x_1 \rangle = 0 \Rightarrow$ ce se întâmplă cu $u_k = A^k u_0$?

$$u_0 = c_1 x_1 + \dots + c_n x_n$$

$$A^k u_0 = c_2 \lambda_2^k x_2 + \dots + c_n \lambda_n^k x_n$$

$\hookrightarrow$ devine dominant

=> MPD găsește a doua cea mai mare valoare proprie în modul ! -> cea foarte rar întâlnit în practică

1) Calculăm $\vec{x_1}$ cu MPD plecând de la u_0

2) Scoatem proiecția pe span $\{\vec{x_1}\}$ a lui $\vec{u_0}$ nou $\Leftrightarrow$ $\langle u_0, x_1 \rangle = 0$ $\Leftrightarrow$ $\begin{cases} u_0 = \text{rand}() \\ u_0 = u_0 - \text{proj}_{x_1} u_0 \end{cases}$

$\lambda_2 x_2$) 3) Calculăm $\vec{x_2}$ cu MPD plecând de la $\vec{u_0}$

4) Scoatem proiecția pe span $\{\vec{x_1}, \vec{x_2}\}$ a lui $\vec{u_0}$ nou $\begin{cases} u_0 = \text{rand} \\ u_0 = u_0 - \text{proj}_{x_1} u_0 \\ u_0 = u_0 - \text{proj}_{x_2} u_0 \end{cases}$

($\lambda_3 x_3$)

5) Calculăm $\vec{x_3}$ cu MPD plecând de la $\vec{U_0}$

⋮

La final => toate perechile val-vect. proprii ale matricii

Se poate rescrie:

1) $u_0^{(1)}$ = vect. random normat

$$u_1^{(1)} = \frac{A u_0^{(1)}}{\| A u_0^{(1)} \|} \text{ ----} \Rightarrow \vec{x_1} \text{ vect. propriu dominant asociat val. proprii}$$

(MPD) ↙

2) $u_0^{(2)}$ = vector random normat

$$u_0^{(2)} = u_0^{(2)} - \underbrace{x_1^T \cdot u_0^{(2)} \cdot x_1}_{\text{proiecția}}$$

↓ am șters componenta x_1 din $u_0^{(2)}$

$$u_1^{(2)} = \frac{A u_0^{(2)}}{\| A u_0^{(2)} \|} \quad | \; - - - \; | \Rightarrow \vec{x_2}$$ vect. propriu asociat val. proprii dominante după λ_1

↙ (MPD)

⋮

i) $u_0^{(i)}$ = vect. random normat

$$u_0^{(i)} = u_0^{(i)} - \underbrace{\sum_{j=1}^{i-1} x_j^T \cdot u_0^{(i)} \cdot x_j}$$

am șters componentele $\vec{x_1}, \dots, \vec{x_{i-1}}$ din $u_0^{(i)}$

$$u_1^{(i)} = \frac{A u_0^{(i)}}{\| A u_0^{(i)} \|} \quad | \; - - - \; | \Rightarrow \vec{x_i}$$ al i-lea vector propriu dominant

!!! Atenție! Din cauza reprezentării lui 0, componentele $\{x_1, \dots, x_{i-1}\}$ nu se șterg complet, deci ortogonalizare trebuie făcută la fiecare pas.

Octave → algoritm

```
function [x, l] = Orthogonal_power_method
                  (A, tol, max_steps)

    x = [];
    l = [];
    n = length(A);
    for i = 1:n
        prev_x = rand(n, i);
        step = 0;
        while 1                                   → șterg vectorii proprii
                                                    dominanți găsiți
                                                    până acum
            for j = 1:size(x, 2)                  ┐ ortogo-
                prev_x -= x(:, j)' * prev_x * x(:, j);  ┘ nalizare
            endfor                                  la fiecare pas
            x = A * prev_x;                       ┐
            x = x / norm(x);                      │
            lambda = x' * A * x;                  │ MPD
            err = norm(x - prev_x);               │
            if (++steps == max_steps) || (err < tol)
                l = [l, lambda];                  │
                x = [x, x];                       │
                break;                            │
            endif                                 ┘
            prev_x = x;
        endwhile
    endfor
endfunction
```

Metoda se bazează pe G-S => instabilă numeric. Pt. a o stabiliza și găsi: nu așteptăm $\vec{x_1}$ înainte să facem o iterație pentru $\vec{x_2}$. Metoda putem simultan pt. toate coloanele + ortogonalizare la fiecare pas.
Putem folosi orice factorizare QR în loc de G-S.

Iterarea ortogonală

$U^{(0)} = [u_1 | \dots | u_n] \in \mathbb{R}^{n \times n}$ ← vectori $n \times 1$ random

↳ matrice random de val.

for $k = 1, 2, \dots$

$Q^{(k)} R^{(k)} \leftarrow U^{(k-1)}$

$U^{(k)} \leftarrow AQ^{(k)}$ → după ce am ortogonalizat $U^{(k-1)}$

↳ am aplicat MP

endfor